

Markdown редактор

Пособие для понимания кода и защиты проекта

C# и Avalonia с нуля

Здесь разобрано, что делает приложение, где находятся его основные части и как объяснить их преподавателю. Сначала изучи базовые понятия, затем проследи действия пользователя по коду. Для защиты важнее понимать причины каждого шага, чем помнить все строки наизусть.

Как пользоваться пособием

Первый проход — основы C# и Avalonia. Второй — устройство именно этого приложения с открытыми рядом исходниками. Третий — вопросы, речь и демонстрация. Короткие примеры кода можно читать как перевод обычного действия на язык программы.

Если до защиты остался час, прочитай основной сценарий, архитектуру, ключевые алгоритмы и ограничения, затем проговори речь и выполни план демонстрации. После этого вернись к незнакомым словам в начальных разделах.

Сведения	Значение
Приложение	Markdown редактор
Исходники	MarkdownEditor/MarkdownEditor.slnx
Дата проверки	11 сентября 2026
Автоматические тесты	16 пройдено

1 C sharp и платформа dot NET с нуля

Язык платформа и интерфейс

C# читается «си шарп». Это язык, на котором мы описываем действия программы: прочитать текст, сравнить символы, обработать нажатие кнопки, сохранить файл. Код в файле .cs — обычный текст с правилами записи. Компьютеру нужно перевести его в выполняемую форму.

.NET читается «дот нет». Это платформа: среда выполнения и большой набор готовых средств для работы со строками, файлами, временем и коллекциями. C# — язык инструкций, .NET — средства для их выполнения. Avalonia — библиотека для построения окон и элементов интерфейса поверх .NET. Это три связанные, но разные вещи.

Представь мастерскую. C# — язык, на котором написано задание мастеру. .NET — оборудование и общие инструменты. Avalonia — набор деталей для панели управления: окна, кнопки, поля ввода. Проект приложения соединяет всё это и добавляет собственные правила работы.

SDK — набор для разработчика. В нём есть инструменты сборки, компилятор и средства запуска. Runtime — среда выполнения готового приложения. Для изменения и сборки нужен SDK. У готовой самостоятельной сборки необходимые компоненты .NET могут лежать рядом с приложением.

NuGet — система получения библиотек. PackageReference в .csproj указывает имя и версию готового пакета. ProjectReference связывает проекты внутри решения. Например, приложение использует классы своего Core через ProjectReference. Это не копирование текста исходников вручную.

Как исходники становятся работающей программой

Обычная последовательность такая: исходники C# и разметка Avalonia → сборка проекта → файлы приложения → запуск среды .NET → создание окна → ожидание действий пользователя. В обычной сборке C# сначала получается промежуточный код IL; среда выполнения может преобразовывать его в машинный код через JIT. Для объяснения этих проектов достаточно знать этот путь; отдельная AOT-сборка здесь не показана в настройках проектов.

Программа с окном не читает весь код сверху вниз заново после каждого клика. При запуске она создаёт объекты и подписывает обработчики на события. Затем работает цикл обработки событий: нажали кнопку — вызван соответствующий метод; изменили текст — вызван другой метод.

Файлы .csproj описывают отдельные проекты. Решение .sln объединяет несколько проектов для удобной работы. Каталог bin содержит результаты сборки, obj — промежуточные файлы. Правильное поведение приложения следует в исходниках, а не в сгенерированных файлах этих каталогов.

Переменная и тип данных

Переменная — именованное место для значения. Тип отвечает на вопрос «какие значения здесь допустимы и какие операции с ними можно выполнять». Имя отвечает на вопрос «зачем это значение понадобилось».

```
int errors = 2;
double accuracy = 97.5;
bool isActive = true;
string text = "Привет";
char letter = 'П';
```

int хранит целое число; double — число с дробной частью; bool — true или false; string — строку; char — одну кодовую единицу UTF-16. В обычном русском и английском тексте отдельная буква обычно занимает один char. Однако эмодзи и некоторые сложные символы могут занимать несколько единиц: String.Length не является универсальным счётчиком видимых знаков.

Знак = присваивает значение. Знак == сравнивает два значения. Поэтому errors = 2 означает «записать 2», а errors == 2 — «проверить, равно ли значение двум». Точка с запятой завершает многие инструкции. Строки пишутся в двойных кавычках, отдельные char — в одинарных.

```
var count = 10;
var caption = "Готово";
```

var не означает «любой тип». Компилятор выводит тип из правой части: count будет int, caption — string. После этого нельзя просто записать строку в count. Такой вывод типа сокращает запись, сохраняя проверку типов.

Условия циклы и индексы

```
if (errors == 0)
{
    return "Без ошибок";
}
return "Есть ошибки";
```

Это учебный фрагмент тела метода, возвращающего string. if означает «если». Условие в круглых скобках должно дать bool. Фигурные скобки объединяют инструкции. return возвращает результат и сразу прекращает текущий вызов метода. В методе void можно написать просто return, чтобы выйти без значения.

```
for (var i = 0; i < text.Length; i++)
{
    var symbol = text[i];
}
```

for повторяет действия. i = 0 — старт; i < text.Length — условие продолжения; i++ — увеличение на единицу после прохода. Индексы начинаются с нуля: в строке «кот» text[0] — «к», text[1] — «о», text[2] — «т». Обращение за пределы строки вызывает ошибку. foreach перебирает готовые элементы коллекции без ручного управления индексом.

Частые операторы: != — не равно; !flag — противоположное логическое значение; && — одновременно выполнены оба условия; || — выполнено хотя бы одно. В выражении a && b правая часть не вычисляется, если a уже false. Тернарная запись condition ? yes : no выбирает одно из двух значений.

Класс объект поле и свойство

Класс — описание устройства определённой сущности. Объект — конкретный экземпляр такого класса в памяти. Один класс настроек позволяет создать несколько объектов с разными настройками. new создаёт экземпляр, а конструктор задаёт его начальное состояние.

```
public sealed class TrainerSettings
{
    public float FontSize { get; set; } = 16f;
    public bool ShowKeyboard { get; set; } = true;
}
```

Это фрагмент реального класса тренажёра. class объявляет класс. public делает его доступным другим частям программы. sealed запрещает создавать классы-наследники от него. FontSize и ShowKeyboard —

свойства. `get` разрешает чтение, `set` — изменение. `= 16f` задаёт начальное значение `float`; суффикс `f` уточняет тип числа. У свойства `bool` без явного начального значения исходное значение `false`.

```
var settings = new TrainerSettings();
settings.FontSize = 24;
```

Первая строка создаёт объект. Вторая меняет его свойство. Точка означает обращение к члену объекта. Поле хранит данные непосредственно внутри класса; свойство предоставляет доступ к данным и при необходимости выполняет дополнительную логику. Поэтому свойство может не хранить отдельное значение, а вычислять его при чтении.

```
public bool IsDirty => text != savedText;
```

Это реальное вычисляемое свойство редактора: «есть несохранённые изменения, если текущий текст отличается от сохранённого». `=>` здесь сокращает запись тела свойства. У свойства нет `set`: присвоить ему `true` вручную нельзя. Нужно изменить `text` или `savedText`, и результат сравнения изменится.

`private` разрешает обращаться к члену только внутри содержащего типа. `public` открывает доступ извне. `internal` ограничивает доступ сборкой. `readonly` запрещает переписывать поле после допустимой инициализации; но если поле содержит список, это само по себе не запрещает добавлять элементы в список. `const` задаёт константу времени компиляции.

Метод и конструктор

Метод — именованное действие. Параметры — входные данные, возвращаемое значение — результат. Вызов `session.Start(text, true)` означает «попросить объект `session` начать попытку с данным текстом и учётом регистра». Тип `void` означает, что метод не возвращает значение.

```
public void Reset()
{
    IsActive = false;
    IsComplete = false;
}
```

Это сокращённый учебный фрагмент сброса состояния. `Reset` — имя метода, пустые круглые скобки означают отсутствие параметров. В настоящем методе тренажёра также очищаются строки и время. Конструктор носит имя класса, не имеет типа возвращаемого значения и вызывается при `new`.

В коде встречается современная запись конструктора прямо возле имени класса: `TypingSession(TimeProvider? timeProvider = null)`. Это первичный конструктор. Параметр со значением по умолчанию разрешено не передавать. Он нужен для выбора источника времени, в том числе тестового.

Наследование интерфейсы и основы ООП

ООП — способ организовать программу вокруг объектов, их данных и действий. На защите полезно объяснить три идеи на конкретных примерах. Инкапсуляция: внутреннее состояние меняется через методы и свойства с контролируемым доступом. Наследование: `MainWindow : Window` расширяет готовое окно `Avalonia`. Полиморфизм: код может работать с разными реализациями одного интерфейса через общий договор.

Интерфейс перечисляет возможности, которые обязан предоставить объект. `IEditorDialogs` описывает выбор файла и вопросы пользователю. Настоящий `EditorDialogs` показывает окна, а `FakeDialogs` в тестах заранее возвращает ответы. `EditorViewModel` обращается к договору `IEditorDialogs`, поэтому его легче проверять без ручных кликов.

Композиция означает, что объект содержит другие объекты: `MainWindow` хранит `TypingSession` или `EditorViewModel`. Здесь обычно не нужно наследовать окно от «сессии»; окно использует сессию. Это хороший пример выбора отношения «имеет» вместо отношения «является».

`override` заменяет разрешённую виртуальную реализацию базового класса. Например, `App` переопределяет `Initialize`. `base` обращается к базовой реализации. `partial` означает, что определение класса разделено на части, в том числе исходную и сгенерированную из разметки.

Коллекции и LINQ

`List<T>` — изменяемый список элементов типа `T`. `List<TypingSessionStats>` хранит результаты попыток. `Dictionary<TKey, TValue>` сопоставляет ключу значение: `Dictionary<char, Border>` позволяет найти изображение клавиши по символу. Здесь `Dictionary` — структура данных C#, а `WordDictionary` — собственный класс приложения, несмотря на похожие названия.

`ReadOnlyList<T>` предоставляет чтение списка через данный интерфейс. `Queue<T>` — очередь: первый добавленный элемент извлекается первым. Она используется при обходе папок. Запись `[]` в подходящем контексте создаёт пустую коллекцию. Угловые скобки задают параметр типа; это называется обобщением, или `generics`.

LINQ — набор операций над последовательностями. `Where` отбирает элементы, `Select` преобразует каждый элемент, `OrderByDescending` сортирует по убыванию, `Take` оставляет первые `N`, `ToList` материализует результат в список. `Max` находит максимум, `Average` — среднее, `GroupBy` собирает элементы в группы.

```
statistics.OrderByDescending(item => item.Date)
    .Take(500)
    .ToList();
```

Это цепочка из тренажёра. Сначала самые новые результаты оказываются впереди, затем сохраняются только первые 500, затем получается отдельный список. `item => item.Date` — лямбда: маленькая функция «для данного элемента взять его дату». `=>` может обозначать лямбду или краткое тело члена класса; смысл определяется контекстом.

Пустое значение и короткие записи

`null` означает отсутствие объекта или значения ссылки. Пустая строка `""` существует и имеет длину ноль; `null` означает, что строки нет. `string?` показывает, что отсутствие допустимо. Настройка `Nullable` в `.csproj` помогает компилятору предупреждать об опасных обращениях.

`InputBox.Text ?? ""` читается «взять `Text`, а если там `null`, использовать пустую строку». Выражение `document?.Path` читает `Path` только при наличии `document`, иначе даёт `null`. Знак `!` после выражения, например `Inlines!`, подавляет предупреждение компилятора о `null`, но не создаёт объект и не защищает во время выполнения.

`is` проверяет тип или шаблон. `is not WordDictionary text` означает «если значение не удалось распознать как `WordDictionary` и получить переменную `text`». `switch` выбирает ветку по значению или типу. `nameof(IsDirty)` даёт строку с именем члена; при переименовании инструменты могут обновить ссылку безопаснее, чем обычный текст в кавычках.

`$"Ошибок: {errors}"` вставляет значение в строку. Формат `:F1` показывает одну цифру после десятичного разделителя, `:F0` — число без дробной части. Это меняет отображение, а не сам расчёт. `\n` в строке означает перевод строки. `//` начинает комментарий для человека.

События асинхронность и освобождение ресурсов

Событие — уведомление о том, что что-то произошло. Обработчик — код, который реагирует. Запись `button.Click += ...` подписывает обработчик; `-=` снимает подписку. Подписка не нажимает кнопку немедленно. В выражении `(_, _) => StartSession()` параметры события не используются, а после события вызывается `StartSession`.

`async` и `await` позволяют описать ожидание операции без блокирующего ожидания потока в точке незавершённого `await`. `Task` представляет выполняемую или завершённую операцию; `Task<string>` дополнительно даёт строковый результат. `async` сам по себе не создаёт отдельный поток. `Task.Run` обычно отправляет вычислительную работу в пул потоков.

```
var document = await Task.Run(
    () => MarkdownDocument.Parse(text));
```

Это фрагмент редактора: разбор Markdown запускается в фоне, а после ожидания результат используется для обновления интерфейса. Элементы Avalonia создают и меняют в UI-потоке — потоке интерфейса. `Dispatcher` помогает выполнить работу в этом потоке. Нельзя считать, что любую операцию с кнопками допустимо делать внутри `Task.Run`.

Исключение — сигнал о невозможности нормально выполнить действие, например прочитать недоступный файл. `try` содержит попытку; `catch` обрабатывает подходящую ошибку; `finally` выполняет завершающие действия при выходе из блока. `throw` создаёт или передаёт исключение дальше. Ловить исключения полезно там, где можно объяснить проблему пользователю или восстановить состояние.

`using` в начале файла подключает пространство имён для коротких имён типов. `using var stream = ...` внутри метода имеет другой смысл: освободить ресурс при выходе из области действия. `IDisposable` и `Dispose` нужны, например, для потоков файлов и изображений. Сборщик мусора освобождает управляемую память, но своевременное закрытие файла всё равно требует правильного управления ресурсом.

2 Avalonia и устройство окна

Что описывает AXAML

Avalonia — средство создания графического интерфейса. Файл `.axaml` использует синтаксис XML для описания дерева элементов. Это декларативная запись: мы говорим, какие элементы нужны и как они размещены. Код `.axaml.cs` описывает связанные с окном действия. Их связывает `x:Class`, а загрузка разметки выполняется при `InitializeComponent`.

```
<Button x:Name="StartButton"
    Content="Начать" />
```

`Button` — кнопка. `x:Name` — имя для обращения из C#. `Content` — содержимое кнопки. Запись `/>` закрывает элемент без вложенных элементов. `xmlns` объявляет пространство имён XML: оно помогает понять, из какой библиотеки пришёл тип. URL в `xmlns` не означает, что приложение загружает разметку с сайта при каждом запуске.

`Window` — окно; `Grid` — сетка; `StackPanel` размещает элементы последовательно; `TextBlock` показывает текст; `TextBox` принимает ввод; `Button` выполняет действие; `ComboBox` предлагает выбор; `CheckBox` хранит состояние флажка; `ScrollView` даёт прокрутку. `Border` добавляет фон, рамку и внутренние отступы. `TreeView` отображает дерево, `TabControl` — вкладки.

Как работает размещение

```
<Grid ColumnDefinitions="*,340">
    <TextBlock Text="Рабочая область" />
    <TextBlock Grid.Column="1" Text="Настройки" />
</Grid>
```

Учебный пример повторяет принцип тренажёра: справа фиксированная ширина 340, слева оставшееся место. * означает долю свободного пространства, Auto — размер по содержимому, число — фиксированный размер в логических единицах интерфейса. Столбцы и строки нумеруются с нуля. Поэтому Grid.Column="1" означает второй столбец.

Margin — внешний отступ, Padding — внутренний отступ между границей и содержимым. IsVisible скрывает или показывает элемент; IsEnabled разрешает или блокирует взаимодействие; IsReadOnly у TextBox запрещает редактирование. TextWrapping="Wrap" переносит длинные строки. Минимальные размеры окна задаются через MinWidth и MinHeight.

Styles задают повторяющееся оформление по селектору. Например, стиль Button меняет все подходящие кнопки, а Classes="tool" позволяет выделить отдельную группу. FluentTheme предоставляет базовый внешний вид элементов. Тема не отвечает за алгоритмы сравнения текста или сохранения файлов.

Привязка данных и MVVM

Binding связывает свойство элемента с данными. DataContext — объект, относительно которого обычно ищут свойства привязки. В редакторе DataContext = ViewModel, поэтому {Binding WindowTitle} получает заголовок из EditorViewModel. x:DataType указывает ожидаемый тип данных для разметки, но сам по себе не создаёт объект и не назначает DataContext.

```
<TextBox Text="{Binding DocumentText, Mode=TwoWay}" />
```

TwoWay означает двустороннюю связь: ввод в поле меняет DocumentText, а изменение DocumentText в модели отражается в поле. Для обновления после изменения обычного свойства модель посылает PropertyChanged через интерфейс INotifyPropertyChanged. Без такого уведомления значение в памяти может измениться, а отображение остаться прежним.

MVVM расшифровывается как Model, View, ViewModel. Model содержит данные и правила предметной области; View — отображение; ViewModel — состояние, свойства и команды, с которыми работает View. Это способ разделить обязанности, а не обязательная библиотека. Наличие .axaml и .axaml.cs ещё не доказывает использование MVVM.

Команда ICommand предоставляет Execute для выполнения и CanExecute для проверки доступности. Один объект команды можно привязать к меню, кнопке и сочетанию клавиш. Code-behind при этом допустим для деталей самого интерфейса: фокус, положение курсора, выделение, закрытие окна.

Путь запуска обоих приложений

Program.Main вызывает BuildAvaloniaApp. AppBuilder.Configure<App>() выбирает класс приложения, UsePlatformDetect() подключает подходящую платформенную реализацию, LogToTrace() включает вывод диагностических сообщений в средства трассировки. StartWithClassicDesktopLifetime запускает жизненный цикл настольного приложения.

App.Initialize загружает App.axaml с темой и стилями. OnFrameworkInitializationCompleted создаёт MainWindow и назначает его главным окном. Конструктор MainWindow загружает разметку, готовит данные

и подписки на события. Затем окно ожидает действий пользователя. Атрибут [STAThread] задаёт модель главного потока, нужную в том числе для совместимости с некоторыми механизмами Windows.

Avalonia позволяет строить приложения для разных платформ, однако эти проекты настроены как настольные приложения. Сам факт применения Avalonia не означает, что уже создана и проверена версия для Android или iOS. Проверки в этом пособии относятся к данному окружению Windows.

3 Как устроен Markdown редактор

Задача приложения и Markdown простыми словами

Markdown — способ добавить оформление в обычный текст с помощью небольшого количества специальных символов. Например, # в начале строки обозначает заголовок, а **слово** — жирное выделение. Сам файл остаётся текстовым: в нём нет сложного устройства Word-документа. Расширения, принимаемые редактором, — .md и .markdown.

```
# План проекта

Это важная заметка и пояснение.

- Открыть файл
- Проверить предпросмотр
- Сохранить результат

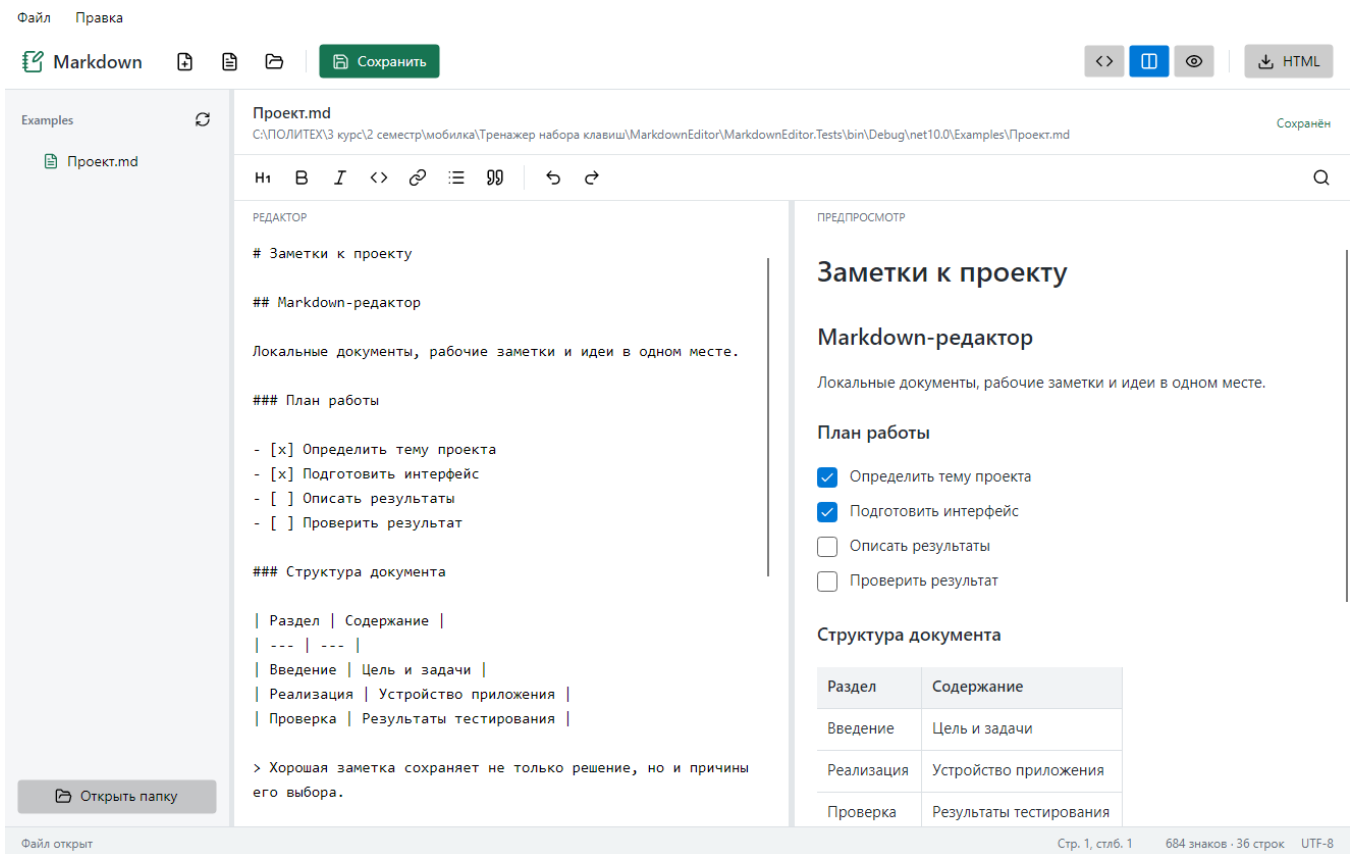
> Небольшая цитата

[Ссылка](https://example.com)
```

В центре редактора пользователь видит именно исходную разметку. Справа приложение строит её оформленное представление. Кнопка жирного текста вставляет звёздочки в исходник, а не применяет форматирование Word. Этот принцип нужно первым показать преподавателю.

Приложение открывает папку, показывает дерево Markdown-файлов, редактирует один документ, ищет текст в документе или папке, сохраняет изменения и экспортирует HTML. Дополнительно есть отмена и повтор, форматирование, три режима отображения, горячие клавиши и предупреждения о несохранённой работе.

HTML — язык разметки веб-страниц. Экспорт превращает Markdown в отдельный HTML-файл, который можно открыть браузером. Сохранение .md и экспорт .html — разные операции: HTML не становится новым исходным документом и не снимает признак несохранённых правок Markdown.



Интерфейс из автоматического теста: слева дерево файлов, в центре исходный текст, справа предпросмотр. Верхняя панель содержит файловые действия и режимы отображения, панель над текстом — вставку разметки. Внизу показаны состояние, положение курсора, количество знаков и строк, кодировка.

Карта решения и библиотек

Решение находится в MarkdownEditor/MarkdownEditor.slnx. MarkdownEditor.App — окно, диалоги и визуальный предпросмотр. MarkdownEditor.Core — состояние документа, команды, файловые операции, поиск и обработка Markdown. MarkdownEditor.Tests — проверки. Examples — документы для демонстрации. SPEC.md содержит требования, README.md — возможности, ограничения и запуск.

Файл	Главное назначение
MarkdownEditor.App/Program.cs	Настройка и старт приложения
MarkdownEditor.App/App.axaml.cs	Главное окно и открытие аргумента командной строки
MarkdownEditor.App/MainWindow.axaml	Панели поля меню и привязки
MarkdownEditor.App/MainWindow.axaml.cs	Фокус форматирования режимы и запуск предпросмотра
MarkdownEditor.App/EditorDialogs.cs	Выбор пути и вопросы пользователю
MarkdownEditor.App/MarkdownPreview.cs	Построение элементов Avalonia из Markdown
MarkdownEditor.Core/EditorViewModel.cs	Состояние свойства команды и координация действий

Файл	Главное назначение
MarkdownEditor.Core/DocumentFiles.cs	Чтение кодировки запись и проверка конфликта
MarkdownEditor.Core/WorkspaceFiles.cs	Дерево папок и текстовый поиск
MarkdownEditor.Core/MarkdownDocument.cs	Настройка Markdig разбор и экспорт HTML

В App.csproj указаны net10.0, Avalonia 12.1.2, FluentTheme 12.1.2 и Lucide.Avalonia 0.2.20. В Core.csproj указан Markdig 1.3.2. Это фактические версии в исходниках. Lucide даёт значки и не выполняет редактирование. Markdig разбирает Markdown, а собственный код связывает всё с интерфейсом и файлами.

Core не ссылается на Avalonia. Он содержит EditorViewModel и договор IEditorDialogs, но не создаёт настоящие окна. App ссылается на Core. Tests ссылается на App и через него использует компоненты приложения.

Разделение по MVVM

View — MainWindow.axaml и код визуального поведения в MainWindow.axaml.cs. ViewModel — EditorViewModel. Модель и сервисы — структуры документа, DocumentFiles, WorkspaceFiles и MarkdownDocument. В проекте MVVM реализован вручную: отдельного ReactiveUI или CommunityToolkit.Mvvm в ссылках проекта нет.

ViewModel знает текущий текст, путь, состояние сохранения и доступные команды. Окно знает, как поставить курсор, выделить найденное слово и переключить панели. Такое разделение позволяет тестировать большинство сценариев с FakeDialogs, не заставляя тест взаимодействовать с настоящим системным выбором файла.

Важный путь: ввод пользователя → TwoWay Binding → DocumentText в EditorViewModel → PropertyChanged → обновление заголовка и состояния → обработчик окна планирует новый предпросмотр. Другой путь: Ctrl+S → SaveCommand → SaveInternalAsync → DocumentFiles.WriteAsync → обновление savedText → исчезновение звёздочки.

Создание окна и открытие первого файла

Program.Main и App запускают настольный жизненный цикл. В App.OnFrameworkInitializationCompleted создаётся MainWindow. После события Opened проверяется первый аргумент командной строки; если путь передан, вызывается window.ViewModel.OpenPathAsync(path). Наличие аргумента не отменяет проверки допустимого расширения и чтения файла.

MainWindow вызывает InitializeComponent, создаёт настоящие диалоги либо принимает переданные тестовые, создаёт EditorViewModel и записывает DataContext = ViewModel. Затем создаёт MarkdownPreview и подписывается на изменения модели, курсора, размера панели и закрытие окна.

Два таймера выполняют разные задачи: 180 мс — задержка предпросмотра после изменений; 220 мс — задержка поиска. Это не обещание, что тяжёлый документ будет полностью обработан за 180 мс. Интервал задаёт паузу до начала очередного обновления.

Состояние документа

Главные внутренние значения EditorViewModel: text — текст сейчас; savedText — текст последней принятой загрузки или успешной записи; currentPath — путь либо null для нового документа; encoding — кодировка; fingerprint — отпечаток последней известной версии файла на диске.

```
public bool IsDirty => text != savedText;  
public string DisplayName => FileName + (IsDirty ? " *" : "");
```

IsDirty означает «текущий текст отличается от сохранённого». Если напечатать букву, а затем отменить изменение до исходного текста, признак снова станет false. Звёздочка — только видимый результат проверки. У нового пустого документа IsDirty может быть false, но SaveState всё равно покажет «Не сохранён», поскольку пути ещё нет.

DocumentText — свойство с дополнительной логикой. При записи нового текста оно сравнивает старое и новое значение, сохраняет новое, отправляет Changed для зависимых свойств и отменяет старый поиск. В числе обновлений — IsDirty, DisplayName, WindowTitle, Statistics и SaveState. Changed вызывает событие PropertyChanged.

Statistics считает text.Length и количество символов '\n' плюс один. Поэтому пустой документ имеет одну строку. Это число логических строк, а не число визуальных переносов по ширине окна. Положение курсора также вычисляется по переводам строк, а не по строкам, появившимся из-за TextWrapping.

Команды и занятость

AsyncCommand реализует ICommand. В конструкторе ему передаются действие Func<Task> и необязательная проверка Func<bool>. CanExecute возвращает доступность, Execute выполняет действие через await, Refresh посылает CanExecuteChanged, чтобы элементы интерфейса заново проверили доступность.

Например, SaveCommand вызывает RunAsync и SaveInternalAsync. К одной команде привязаны кнопка, меню и сочетание клавиш. Это избавляет от трёх разных реализаций сохранения. AsyncCommand.Execute имеет async void из-за сигнатуры ICommand; основную асинхронную работу удобнее держать в методах, возвращающих Task, как и сделано здесь.

RunAsync не запускает новую файловую операцию, если IsBusy уже true. На время действия он устанавливает IsBusy = true, затем в finally возвращает false. IsEditable вычисляется как !busy. Окно и команды блокируют редактирование и конфликтующие действия на время файловой операции.

RunAsync перехватывает предусмотренные ошибки чтения, доступа, аргументов и неподдерживаемых операций, меняет Status и показывает сообщение через IEditorDialogs. Это центральное место обычной обработки ошибок. Такой обработчик не означает, что абсолютно любое исключение во всём приложении будет безопасно обработано.

Открытие файла по шагам

1. Пользователь выбирает файл в системном диалоге либо дереве. При отмене диалог возвращает null, действие прекращается.
2. OpenInternalAsync проверяет путь: повторное открытие уже текущего пути пропускается, расширение должно быть .md или .markdown.

3. `MayDiscardAsync` выясняет, можно ли заменить текущий документ. Если есть изменения, требуется решение пользователя.

4. `DocumentFiles.ReadAsync` читает файл, определяет кодировку, получает текст и SHA-256 отпечаток байтов. Только после успешного чтения вызывается `SetDocument`.

5. `SetDocument` одновременно устанавливает путь, текущий и сохранённый текст, кодировку и отпечаток. Отправляет уведомления и `DocumentLoaded`. Окно очищает историю Undo, возвращает курсор и предпросмотр в начало.

Почему сначала читаем, а потом заменяем состояние? Если файл не существует или чтение не удалось, предыдущий текст должен остаться в памяти. В тестах проверяется именно это. Очистка Undo при новом документе предотвращает восстановление текста прошлого файла в новом документе.

Несохранённые изменения и закрытие

`MayDiscardAsync` при `IsDirty = false` сразу разрешает действие. Иначе показывает три варианта: «Сохранить», «Не сохранять», «Отмена». «Не сохранять» разрешает продолжение; «Отмена» запрещает; «Сохранить» разрешает продолжение только если сохранение действительно завершилось успешно.

Ключевой случай: пользователь выбрал «Сохранить», а затем отменил выбор имени файла. Это не является успешным сохранением. `SaveInternalAsync` возвращает `false`, поэтому новый документ, открытие другого файла или закрытие не продолжаются. Правки остаются на месте.

При закрытии окна обработчик сначала ставит `e.Cancel = true`, потому что решение требует асинхронного диалога. Если `CanCloseAsync` разрешил закрытие, `allowClose` становится `true` и `Close` вызывается ещё раз. `closingPrompt` защищает от нескольких одновременных диалогов. Так обычное закрытие окна согласовано с асинхронным вопросом пользователю.

`EditorDialogs.ShowAsync` возвращает безопасное значение отмены при закрытии диалога крестиком. Это важно для enum `SaveDecision`: первый элемент `Save` имеет числовое значение по умолчанию, но закрытие крестиком не должно случайно трактоваться как «Сохранить».

Чтение байтов и кодировка

Файл на диске — последовательность байтов. Кодировка объясняет, как превратить байты в буквы и обратно. Неправильная кодировка может испортить кириллицу. BOM — необязательная последовательность байтов в начале файла, по которой можно распознать определённую кодировку и порядок байтов.

`DocumentFiles` поддерживает UTF-8 без BOM и с BOM, UTF-16 и UTF-32 при наличии распознаваемого BOM. Без BOM предполагается строгий UTF-8. Windows-1251 автоматически не определяется. Если последовательность байтов недопустима для выбранной кодировки, чтение отклоняется, а не подменяет молча буквы.

Проверка UTF-32 выполняется раньше UTF-16, потому что некоторые сигнатуры начинаются одинаково. Метод `DetectEncoding` возвращает кодировку и количество начальных байтов, которые нужно пропустить перед декодированием. При записи используется сохранённая кодировка вместе с её преамбулой.

Лимит документа — $4 \times 1024 \times 1024$ байт. Он проверяется и перед чтением, и во время чтения частями по 8192 байта: файл мог увеличиться после первой проверки. Это ограничение на размер в байтах, а не на четыре миллиона букв. Перед записью также проверяется размер получившихся байтов.

LoadedDocument — record с Path, Text, Encoding и Fingerprint. Record удобен как контейнер связанных данных и по умолчанию поддерживает сравнение по значениям своих членов. Само слово record не превращает всё содержимое, включая вложенные объекты, в абсолютно неизменяемое.

Сохранение и защита от чужих изменений

SaveInternalAsync выбирает целевой путь. Для нового документа или «Сохранить как» открывает диалог. Проверяет расширение; при выборе другого существующего файла спрашивает о замене. Затем берёт snapshot текущего текста: конкретное содержимое, которое собирается записать.

DocumentFiles.WriteAsync кодирует snapshot в байты и записывает их в уникальный временный файл в том же каталоге. Прежде чем заменить оригинал, он сравнивает его текущий отпечаток с expectedFingerprint, если тот передан. Отпечаток — SHA-256 от байтов, а не дата изменения.

```
if (expectedFingerprint is not null && !overwriteConflict &&
    (!File.Exists(path) ||
     !await MatchesFingerprintAsync(path, expectedFingerprint)))
    throw new FileConflictException();
```

Это реальный фрагмент проверки. Если ожидалась прежняя версия, а файла уже нет или его байты отличаются, создаётся FileConflictException. ViewModel перехватывает это специальное исключение и спрашивает пользователя, хочет ли он заменить файл текущим текстом.

Если пользователь отменил замену, внешний файл остаётся на диске, а текущие правки сохраняются в памяти как несохранённые. Если согласился, запись повторяется с разрешением замены. При успехе обновляются currentPath, fingerprint и savedText, после чего исчезает звёздочка и обновляется дерево выбранной папки.

SHA-256 здесь используется как отпечаток для сравнения версий. Это не шифрование текста и не механизм восстановления предыдущей версии. Файл хранится обычным текстом. Проверка уменьшает риск случайно затереть чужие изменения, но между сравнением и заменой остаётся небольшое окно гонки. Блокировки для полноценного совместного редактирования здесь нет.

Временный файл защищает от преждевременного обрезания оригинала при ошибке подготовки записи. Его остаток удаляется в finally. Нельзя называть это полной системой транзакций или резервного копирования. Автосохранения и постоянной истории версий в проекте нет.

Как Markdig превращает разметку в структуру

MarkdownDocument.Pipeline создаётся через MarkdownPipelineBuilder. Подключены таблицы с вертикальными чертами, списки задач, дополнительные выделения, автоматические ссылки и идентификаторы заголовков. DisableHtml отключает интерпретацию встроенного HTML. Набор расширений задан явно.

MarkdownDocument.Parse возвращает дерево синтаксических элементов — AST. Это описание смысла частей текста. Например, строка «# Привет» становится HeadingBlock, обычный абзац — ParagraphBlock, жирный фрагмент — EmphasisInline внутри строки. Блоки описывают крупные части, inline-элементы — содержимое внутри них.

Дальше есть два направления. Для предпросмотра MarkdownPreview обходит AST и создаёт элементы Avalonia. Для экспорта Markdig формирует HTML, который помещается в готовый документ с заголовком,

стилями и метаданными. Предпросмотр не является встроенным браузером и не отображает HTML-страницу через WebView.

Предпросмотр и защита от устаревшего результата

При каждом изменении `SchedulePreview` увеличивает `renderVersion`, останавливает и вновь запускает таймер. Если печатать быстро, таймер постоянно откладывается до короткой паузы. Такой приём называется `debounce`: дорогая операция не запускается на каждое мгновенное изменение.

`RefreshPreviewAsync` получает очередной номер версии и снимок текста, затем вызывает разбор в `Task.Run`. Пока разбор идёт, пользователь может уже изменить текст. После `await` метод сравнивает свою версию с `renderVersion`. Если версия устарела или окно закрыто, результат не показывается.

```
var version = ++renderVersion;
var text = ViewModel.DocumentText;
var document = await Task.Run(() => MarkdownDocument.Parse(text));
if (disposed || version != renderVersion) return;
preview.Render(document, ViewModel.CurrentPath,
    PreviewPane.Bounds.Width);
```

Это фрагмент основного алгоритма. Представь, что старый большой текст разбирался дольше нового короткого. Проверка номера не позволит позднему старому результату заменить актуальный. При этом работа старого парсера не обязательно прерывается: её результат просто отбрасывается. Отмена поиска устроена иначе, через `CancellationToken`.

`MarkdownPreview.Render` очищает прежние визуальные элементы, освобождает старые `Bitmap`, сбрасывает якоря заголовков и заново создаёт элементы. Заголовки становятся `SelectableTextBlock` увеличенного размера, абзацы — текстовыми блоками, цитаты — `Border` с боковой линией, списки — панелями строк, код — моноширинным текстом с прокруткой, таблицы — `Grid` внутри `ScrollView`.

`Fill` обрабатывает содержимое строк. `LiterallInline` даёт `Run` с обычным текстом, `EmphasisInline` — `Span` с выделением, `CodeInline` — моноширинный `Run`, ссылки — кнопки, помещённые в `InlineUIContainer`. `Span` объединяет фрагменты с общим оформлением. Списки задач имеют флажки только для показа: щелчок не редактирует исходник.

Ссылки изображения и HTML

Ссылка вида `#заголовок` ищется в словаре `anchors`, который связывает идентификатор с элементом управления. `BringIntoView` прокручивает его в видимую область. Локальная ссылка на `Markdown` открывается как документ приложения. `http`, `https` и `mailto` передаются системному `Launcher`. Другие неподдерживаемые варианты вызывают сообщение.

Локальное растровое изображение загружается из файла относительно пути документа или по абсолютному пути. Предусмотрен предел 16 МБ и декодирование до ширины 1200. На экране картинка ограничивается шириной панели и высотой 400, пропорции сохраняются. `Bitmap` сохраняются в списке и освобождаются через `Dispose` при следующем обновлении или закрытии.

Сетевые изображения автоматически в предпросмотр не скачиваются. Вместо недоступной картинки появляется кнопка с подписью. Для веб-адреса её нажатие открывает ссылку через браузер. Несохрани́нный документ не имеет исходной папки для разрешения относительного локального пути: сначала полезно сохранить `.md` рядом с ресурсами.

Экспорт получает HTML через Markdig, экранирует заголовок WebUtility.HtmlEncode, добавляет UTF-8, CSS и Content-Security-Policy. Встроенный HTML исходника не исполняется как произвольная разметка. CSP ограничивает ресурсы и скрипты, однако это не обещание абсолютной безопасности любых внешних ссылок и любых браузеров.

Если исходный документ сохранён, относительные адреса ресурсов при экспорте приводятся к абсолютным URI относительно исходного пути. Это помогает открыть экспорт на том же компьютере из другой папки. Изображения внутри HTML не упаковываются: для переноса нужно перенести ресурсы и проверить адреса. Абсолютный локальный путь сам по себе не переносим на чужой компьютер.

Дерево папок и поиск

WorkspaceFiles.ReadTree рекурсивно строит FileNode. У узла есть Name, FullPath, IsDirectory и Children. Для папки Children содержит вложенные элементы, для файла — пустой список. TreeDataTemplate в AXAML повторяет один шаблон отображения для каждого уровня дерева.

В дереве предусмотрен бюджет около 10 000 просматриваемых элементов и ограничение глубины около 30 уровней. Это защитные пороги, а не обещание показать ровно 10 000 строк. Скрытые, системные, недоступные элементы и parse points пропускаются; каталоги bin, obj и node_modules исключены. Папки и Markdown-файлы сортируются по именам.

Поиск по папке использует отдельный обход EnumerateMarkdown с очередью Queue. В отличие от построения дерева, у этого обхода нет того же бюджета 10 000 и глубины 30. Он может найти файл, который не попал в ограниченное дерево. Итоговый список совпадений ограничен 500 и поддерживает отмену.

Find выполняет буквальный поиск через IndexOf с учётом регистра или без него. Это не регулярные выражения. После совпадения смещение увеличивается на длину запроса, поэтому перекрывающиеся совпадения пропускаются. Например, запрос «aa» в «aaa» даст одно совпадение.

SearchHit хранит путь, имя файла, строку, столбец, Offset, Length и Snippet. Offset начинается с нуля и удобен для выделения в TextBox; строка и столбец показываются пользователю с единицы. Snippet — короткий фрагмент текста вокруг результата.

```
Текст:  абв\r\nтест тест
Запрос:  тест
Результат 1: строка 2, столбец 1, Offset 5
Результат 2: строка 2, столбец 6, Offset 10
```

Такой случай проверяется тестом. `\r\n` содержит две кодовые единицы, поэтому первая «т» находится по индексу 5. При переходе из результата окно выделяет текст через SelectionStart и SelectionEnd, а ScrollToLine использует номер строки с нуля.

Поиск текущего документа работает по тексту в памяти, включая несохранённые изменения. При поиске по папке для уже открытого файла тоже используется его текущий буфер; остальные файлы читаются с диска. Это предотвращает ситуацию, когда пользователь ищет новое слово и не находит его только потому, что ещё не сохранил документ.

SearchAsync отменяет предыдущий запрос через CancellationTokenSource. Новый запрос берёт снимок текста, строки поиска, папки и параметров. При изменении этих данных предыдущие результаты очищаются. Чтобы узнать, есть ли больше 500 совпадений, алгоритм допускает поиск 501-го и показывает 500 с признаком продолжения.

Недоступные, неподходящие по кодировке или слишком большие файлы могут быть пропущены, и их количество отражается в итоге. При открытии результата из другого файла совпадения вычисляются заново и выбирается ближайшее к прежнему смещению: файл мог измениться после поиска. Постоянного индекса, как у большой поисковой системы, здесь нет.

Форматирование отмена и режимы

Format берёт выделенный текст и добавляет prefix и suffix. Для bold это ** с обеих сторон, для italic — *, для code — обратные кавычки, для link — квадратные скобки и адрес. Если выделения нет, вставляется текст-заготовка, который сразу выделяется для замены.

Заголовок, цитата и список работают с началом строки. Метод находит предыдущий '\n' и вставляет #, > или - с пробелом. При многострочном выделении префикс добавляется также после переводов строк. Это простые операции со строкой и выделением, а не полноценное структурное редактирование AST.

Undo и Redo предоставлены самим TextBox Avalonia. Проект не реализует собственный стек истории. При загрузке другого документа IsUndoEnabled временно выключается и включается, чтобы очистить историю старого текста.

ApplyMode показывает только редактор, только предпросмотр или обе панели. Он меняет IsVisible и ColumnDefinitions. При переходе к результату поиска из режима чтения окно возвращается в раздельный режим, чтобы найденное выделение было видно.

Горячие клавиши: Ctrl+N — новый документ; Ctrl+O — файл; Ctrl+Shift+O — папка; Ctrl+S — сохранение; Ctrl+Shift+S — «Сохранить как»; Ctrl+E — HTML; Ctrl+F — поиск; Ctrl+B и Ctrl+I — выделение в редакторе; Esc — закрыть поиск. Отмена и повтор используют функции TextBox и кнопки интерфейса.

4 Проверка запуск и развитие редактора

Что проверяют автоматические тесты

11 сентября 2026 года выполнен MarkdownEditor.Tests: 16 пройденных тестовых случаев, 0 ошибок, 0 пропусков. Число случаев больше числа отдельных методов, поскольку Theory запускается с разными параметрами. Проверки относятся к текущему коду в данном Windows-окружении.

CoreTests проверяет цикл чтение-запись для UTF-8, UTF-8 с BOM и UTF-16, сохранение переводов строк, обнаружение внешних изменений, отмену сохранения перед новым документом или закрытием, сохранение буфера при неудачной операции, отказ от слишком большого и неверного UTF-8 файла, поиск по несохранённому буферу и вложенной папке, координаты результатов, экспорт без выполнения встроенного script и разрешение путей ресурсов.

WindowTests проверяет ввод, форматирование, Undo, сохранение, выделение результата поиска, сброс истории нового документа, предпросмотр и режимы при 1320 × 840 и 1000 × 640, локальную картинку, а также отмену настоящего диалога его крестиком. Для интерфейса используется Avalonia.Headless.XUnit.

FakeDialogs реализует IEditorDialogs и возвращает заранее заданные пути и решения. Например, тест задаёт SavePath = null и Decision = Save. Это моделирует пользователя, который сначала хочет сохранить, но затем отменяет диалог имени файла. Проверка убеждается, что документ не потерян.

Набор тестов не проверяет каждую возможную кодировку и каждое сочетание Markdown. В частности, наличие обработки UTF-32 в коде не означает отдельную параметризованную проверку всех вариантов UTF-

32 этим набором. На защите лучше привести конкретный проверенный сценарий, чем говорить «протестировано абсолютно всё».

Запуск и перенос

Команды ниже выполняются из папки MarkdownEditor. Для сборки README требует .NET SDK 10. Открыть решение можно в подходящей среде разработки, а команды выполнить в терминале.

```
dotnet restore MarkdownEditor.slnx --configfile NuGet.Config
dotnet run --project MarkdownEditor.App
dotnet test MarkdownEditor.Tests
```

Start.cmd сначала использует готовую сборку, если она имеется, иначе запускает проект через SDK. Перед защитой нужно проверить именно тот способ запуска, который будет использован перед преподавателем.

```
dotnet publish MarkdownEditor.App -c Release -r win-x64 --self-contained true -o bin/Release/win-x64
```

По README готовое приложение находится в bin/Release/win-x64/MarkdownEditor.exe. Для переноса копируют всю папку win-x64, а не только .exe. Самостоятельная сборка включает необходимые компоненты .NET, но остаётся сборкой под конкретную платформу и архитектуру Windows x64.

Если предпросмотр картинки пустой, сначала проверь, сохранён ли Markdown и правильный ли относительный путь. Если кириллица не читается, проверь кодировку: Windows-1251 не входит в автоматическое определение. Если файл не открывается, проверь расширение, размер и доступ. Проблемы запуска SDK и проблемы содержимого документа — разные причины.

Ограничения которые важно назвать

Открыт один документ; вкладок с несколькими файлами нет. Нет автосохранения, базы SQL, облачной синхронизации, учётных записей, совместного редактирования и постоянной истории версий. Дерево обновляется кнопкой, загрузкой папки и после сохранения; постоянного наблюдателя за внешними изменениями каталога не реализовано.

Предпросмотр строится собственными элементами Avalonia и может отличаться от HTML в браузере по оформлению и отдельным деталям. Нет полноценного редактирования «как в Word», произвольного встроенного HTML, автоматической загрузки сетевых картинок и подсветки синтаксиса всех языков программирования. Флажки задач в предпросмотре не интерактивны.

Полный текст разбирается заново, а визуальные элементы перестраиваются; у очень сложного документа даже в пределах 4 МБ это может быть дорого. Порог размера не гарантирует мгновенную работу. Поиск буквальный, без замены и регулярных выражений. Защита SHA-256 обнаруживает многие конфликты, но не заменяет блокировку файла.

Куда вносить изменения

Добавить сочетание клавиш для файловой команды: Window.KeyBindings в MainWindow.axaml и, если требуется новое действие, команду в EditorViewModel. Добавить форматирование: кнопку с Tag и ветку Format в MainWindow.axaml.cs. Изменить формулу признака несохранённых правок: IsDirty и связанные уведомления в ViewModel.

Добавить новый вид Markdown: сначала выяснить, поддерживает ли его Pipeline; затем добавить отображение подходящего узла в MarkdownPreview и тест. Изменить лимит документа:

DocumentFiles.MaxBytes и описание ограничения. Изменить поведение экспорта: MarkdownDocument.ExportHtml и при необходимости ExportInternalAsync.

Разумное развитие: автоматическое резервное сохранение с понятным восстановлением; несколько вкладок; наблюдение за папкой; более экономное обновление предпросмотра; упаковка HTML с ресурсами; расширение поиска. Это будущие задачи. Для нескольких вкладок понадобится отдельное состояние на каждый документ и согласование закрытия вкладок с несохранёнными изменениями.

5 Подготовка к защите редактора

Речь примерно на четыре минуты

«Мой проект — настольный Markdown-редактор на C# и Avalonia. Он решает задачу работы с текстовыми документами, в которых оформление задаётся символами Markdown. Слева находится дерево файлов выбранной папки, в центре исходный текст, справа оформленный предпросмотр. Есть сохранение, поиск, HTML-экспорт, отмена действий и защита несохранённых изменений.

Решение разделено на App, Core и Tests. ViewModel хранит текущий документ и команды. View описывает интерфейс и поведение курсора. Файловые операции, поиск и обработка Markdown вынесены отдельно. Связь поля редактора с DocumentText выполнена двусторонней привязкой, а PropertyChanged сообщает интерфейсу о новых значениях.

Когда пользователь печатает, обновляются признак изменений, заголовок и статистика. После короткой паузы Markdig разбирает текст в дерево синтаксических элементов. Самописный MarkdownPreview переводит это дерево в элементы Avalonia. Поэтому предпросмотр не является браузером. Номер версии защищает интерфейс от устаревшего результата фонового разбора.

При сохранении приложение сначала подготавливает временный файл. Если редактируется ранее открытый документ, оно проверяет SHA-256 отпечаток версии на диске. При внешнем изменении требуется решение пользователя о замене. Признак сохранения снимается только после успешной записи. Отмена диалога сохранения не приводит к потере текущего документа.

Поиск доступен в текущем тексте и в папке. Для открытого файла учитываются несохранённые изменения. Результаты содержат координаты, и выбор результата выделяет текст в редакторе. Устаревший поиск отменяется, вывод ограничивается 500 совпадениями.

HTML-экспорт строится отдельно, включает стили и не помечает исходный Markdown сохранённым. Локальные изображения не упаковываются внутрь HTML, поэтому при переносе нужно учитывать ресурсы.

Существующий набор тестов прошёл 16 случаев: файловые операции, конфликты, отмена сохранения, поиск, форматирование, предпросмотр и диалоги. Ограничения проекта — один документ, размер до 4 МБ, ограниченный набор кодировок и отсутствие совместного редактирования. Сейчас покажу редактирование, сохранение, поиск и экспорт, затем связь этих действий с ViewModel».

План демонстрации

1. Открой папку Examples и документ «Проект.md». Покажи дерево, исходник и предпросмотр. Объясни на одной строке, что # обозначает заголовок.
2. В демонстрационной копии добавь строку, выдели слово и нажми жирное форматирование. Покажи звёздочку в имени и появление ** в исходнике. Выполни Undo.

3. Введи уникальное слово «проверказащиты», открой Ctrl+F и найди его до сохранения. Объясни, почему поиск видит текст из памяти.
4. Сохрани демонстрационный файл под новым именем в подготовленной папке. Покажи исчезновение звёздочки и повторное открытие содержимого.
5. Измени текст и начни создавать новый документ. Выбери «Отмена». Докажи, что правки остались. Сценарий внешнего конфликта показывай на отдельной копии, заранее отрепетировав его.
6. Экспортируй HTML и открой результат браузером. Объясни, почему это отдельный файл и почему экспорт не заменяет сохранение Markdown.
7. Покажи Binding DocumentText, свойство IsDirty, SaveInternalAsync и небольшой фрагмент MarkdownPreview. Заверши примером FakeDialogs в тестах.

Частые вопросы и понятные ответы

Что такое Markdown? Обычный текст с простыми обозначениями оформления. Например, # — заголовок, ** — жирное выделение. Файл остаётся текстовым.

Чем отличается Markdown от HTML? Markdown удобен для исходного текста человека; HTML описывает структуру веб-страницы. Экспорт преобразует одно представление в другое.

Что написал разработчик, если есть Markdig? Окно, состояние документа, команды, файлы, защиту сохранения, поиск, нативное отображение AST и интеграцию. Markdig выполняет синтаксический разбор и генерацию HTML.

Где MVVM? MainWindow — View, EditorViewModel — ViewModel, файловые и предметные компоненты Core — модель и сервисы. Связь показана Binding и DataContext.

Зачем PropertyChanged? Чтобы после изменения свойства интерфейс узнал о новом значении и перечитал привязку.

Почему TextBox связан TwoWay? Текст должен идти от модели к полю при открытии файла и от поля к модели при вводе.

Что такое команда? Объект с Execute и CanExecute. Один SaveCommand используется несколькими элементами интерфейса.

Что означает IsDirty? Текущий текст отличается от savedText. Если вернуть исходное содержимое, признак исчезает без записи.

Почему новое пустое окно не обязательно IsDirty? Пока текст равен исходной пустой строке, различий нет. Но пути нет, поэтому отдельный статус сообщает «Не сохранён».

Как работает Undo? Используется встроенная история TextBox, а при смене документа она очищается.

Зачем IEditorDialogs? ViewModel зависит от договора диалогов. В приложении показываются окна, в тестах ответы возвращаются автоматически.

Чем async отличается от отдельного потока? async разрешает await; оно само не создаёт поток. Task.Run используется для фоновых вычислений, например разбора Markdown.

Что такое debounce? Короткое откладывание работы до паузы в серии изменений. Здесь таймер перезапускается при печати.

Почему нужен `renderVersion`? Чтобы старый результат долгого разбора не заменил свежий. Неактуальная версия после завершения игнорируется.

Почему Core без Avalonia, если там ICommand? ICommand — общий интерфейс команд .NET. Конкретные окна Avalonia и реальные диалоги находятся в App.

Что такое AST? Дерево смысла разметки: заголовки, абзацы, списки и внутренние выделения. Preview обходит его и создаёт элементы интерфейса.

Почему нельзя просто показать HTML в браузере внутри окна? Можно было бы выбрать такой подход, но здесь реализован нативный рендерер Avalonia. Он даёт контроль над элементами, но требует самостоятельной поддержки видов Markdown.

Что хранит SHA-256? Короткий отпечаток байтов известной версии файла. По нему обнаруживаются изменения; сам текст не шифруется.

Есть ли полная защита от одновременной записи? Нет. Проверка перед заменой оставляет окно гонки, поэтому полноценное совместное редактирование не заявляется.

Зачем временный файл? Подготовка нового содержимого не обрезает оригинал. Подмена выполняется после успешной записи и проверки конфликта.

Что происходит при отмене выбора имени? `SaveInternalAsync` возвращает `false`, исходное действие не продолжается и правки остаются в памяти.

Как учитывается кириллица? Файл декодируется поддерживаемой Unicode-кодировкой; исходная кодировка запоминается для записи. Windows-1251 автоматически не определяется.

Почему поиск видит несохранённое слово? Для текущего документа берётся буфер `ViewModel`, а не только файл на диске.

Поиск использует регулярные выражения? Нет, `IndexOf` выполняет буквальное сравнение. Перекрывающиеся совпадения не включаются.

Почему дерево и поиск могут показать разные файлы? У построения дерева есть бюджет и глубина; поиск обходит папку отдельно и ограничивает число совпадений, а не тем же бюджетом дерева.

Почему экспорт не снимает звёздочку? Он записывает HTML-представление, а не текущий исходный `.md`. `savedText` при экспорте не меняется.

Можно ли запускать HTML-код из Markdown? Интерпретация встроенного HTML отключена в Pipeline. Ссылки и экспорт имеют отдельные ограничения; это не среда выполнения произвольных скриптов.

Что требуется улучшить в производительности? Полный повторный разбор и перестроение элементов могут быть дорогими. Можно исследовать частичное обновление и ограничение сложности, измерив реальное время.

Самопроверка

Без подсказки проследи один путь: `Ctrl+S` → `SaveCommand` → `RunAsync` → `SaveInternalAsync` → `WriteAsync` → временный файл → проверка отпечатка → замена → `savedText` → `PropertyChanged` → исчезновение звёздочки. Если понимаешь, зачем нужен каждый шаг, можно ответить на большинство вопросов о сохранении.

Затем проследи ввод: `TextBox` → `DocumentText` → уведомление → таймер → `Parse` в фоне → проверка версии → `MarkdownPreview.Render`. Не смешивай состояние текста, разбор разметки и рисование элементов: это разные обязанности.

6 Где проверить объяснения

Главный источник описания приложения — его исходники. Имена классов и методов в тексте позволяют найти соответствующий участок поиском в редакторе кода. Версии библиотек указаны по `.csproj`, результаты тестов — по запуску 11 сентября 2026 года. Учебные фрагменты отдельно названы учебными; основные алгоритмы приведены из проекта.

`MarkdownEditor.Core/EditorViewModel.cs` — команды и правила смены документа.

`MarkdownEditor.Core/DocumentFiles.cs` — кодировка, отпечаток и запись.

`MarkdownEditor.Core/WorkspaceFiles.cs` — дерево и поиск.

`MarkdownEditor.Core/MarkdownDocument.cs` — Pipeline и HTML.

`MarkdownEditor.App/MainWindow.axaml.cs` и `MarkdownPreview.cs` — обновление и показ.

`MarkdownEditor.Tests/CoreTests.cs` и `WindowTests.cs` — проверяемые сценарии.

Официальные материалы для уточнения терминов

Microsoft Learn — обзор языка C#

<https://learn.microsoft.com/en-us/dotnet/csharp/tour-of-csharp/>

Microsoft Learn — асинхронное программирование

<https://learn.microsoft.com/en-us/dotnet/csharp/asynchronous-programming/>

Avalonia — контекст данных

<https://docs.avaloniaui.net/docs/data-binding/data-context>

Avalonia — синтаксис привязки

<https://docs.avaloniaui.net/docs/data-binding/data-binding-syntax>

Avalonia — MVVM

<https://docs.avaloniaui.net/docs/fundamentals/the-mvvm-pattern>

Markdig — официальный репозиторий и описание

<https://github.com/xoofx/markdig>